AI CHEF

ONE-PAGE MASTER BUILD PROMPT | “See it. Cook it.”

Build a polished, production-minded AI cooking assistant called **AI Chef** for **students, bachelors, beginners and everyday home cooks**. Deliver **both a responsive website and an Android app**, in **English only**. The core experience: a user photographs food they see in real life or uploads an image from their gallery; AI identifies the likely dish and creates a practical beginner-friendly recipe to recreate it at home.

1. CORE USER FLOW

Welcome screen → user chooses **Open Camera** or **Choose from Gallery** → preview / retake / remove photo → AI analyzes image → results page shows dish name and confidence → ingredients, quantities and ratios → serving-size adjustment → step-by-step cooking guide → relevant reference-video links. Ask for camera or photo-library permission only when the user selects that action. Include clear loading, empty, permission-denied, network-error and retry states.

2. MUST-HAVE FEATURES

Food recognition: identify the most likely dish, explain uncertainty, and handle non-food or unclear photos gracefully. **Recipe generator:** ingredients with practical metric and kitchen measurements (g, ml, tsp, tbsp, pieces), estimated quantities, ingredient ratios, prep/cook time, difficulty, equipment, alternatives, tips and numbered steps. **Serving calculator:** 1, 2, 3, 4, 5, 10 or custom servings; recalculate all ingredient quantities. **Video discovery:** provide relevant YouTube tutorial search/results links; never invent or claim a video is verified. **Student Budget Mode:** estimated total and per-serving cost, budget-friendly substitutions and local currency support for Bangladesh (৳), while allowing currency configuration. **Personal library:** sign-in, saved recipes, favorites, recent cooking history, search, share and delete.

3. DESIGN SYSTEM

Premium, modern, warm, minimal food-tech aesthetic; mobile-first, responsive, accessible and consistent across web and Android. **Forest Green #244C3B** (primary/buttons), **Warm Ivory #F7F6F1** (page background), **Fresh Lime #D5F078** (accent/important actions), **Charcoal #20231F** (text), white cards, soft sage supporting tones and subtle borders/shadows. Typography: **Playfair Display** for editorial headings and **DM Sans** for body, controls and measurements. Use generous whitespace, rounded cards, high-quality food imagery and restrained motion. Avoid neon colors, clutter and generic template styling. Homepage headline: **“See it. Cook it.”**

4. SCREENS

Website: Home/photo scanner, recipe results, Explore recipes, Budget Meals, Saved Recipes, My Kitchen/Profile, Settings and About. **Android:** equivalent core screens with native camera/gallery, Android back navigation, permission guidance and reliable network handling. Keep the main camera and gallery actions visible immediately on mobile.

5. TECHNICAL ARCHITECTURE

Use **Next.js + React + TypeScript** for the website and secure API; **React Native + Expo** for Android; a shared recipe data model and one backend for both clients; **Gemini Vision** (or a configurable vision model) for image analysis and structured recipe generation; **Supabase** for authentication, database and saved recipes; **YouTube Data API** or reliable search links for video discovery; **Vercel** for web deployment and **Expo EAS** for Android builds. Keep secrets server-side. Provide environment-variable examples, setup instructions and deployment steps.

6. ACCURACY, SAFETY & QUALITY

A photo cannot reveal hidden ingredients or exact original quantities: label measurements as **estimates**, separate visible observations from inferred recipe ingredients, and show confidence. Include allergy prompts/warnings, food-safety advice and safe cooking temperatures where relevant. Validate image type/size, add API rate limiting, protect user data, allow deletion, and never expose API keys in frontend/mobile code. Every visible control must work; no fake buttons, placeholder recipe results or fabricated video links.

7. DELIVERY EXPECTATION

Build a working end-to-end MVP, not just a mockup: complete source code for both clients, shared backend/API integration, camera and gallery upload, real AI response handling, recipe results UI, setup README, .env.example, responsive layouts, graceful error states and Android build configuration. If credentials or external services are unavailable, clearly mark the exact setup needed and provide a functional demo mode that is visibly labelled as demo. Build in phases: (1) core scan-to-recipe flow, (2) serving/budget/video features, (3) accounts and saved recipes, (4) Android release testing and deployment.

IMPORTANT: Prioritize a complete, reliable scan-to-recipe journey before adding decorative features. Use the same backend and recipe format for the website and Android app. Do not claim deployment or APK completion until actually built and tested.